HANDLING CONCURRENCY - AGGREGATE PATTERN AND EF CORE

2020-05-14 ARCHITECTURE & DESIGN DOMAIN-DRIVEN DESIGN DATABASE .NET



Introduction

In this post I would like to discuss a frequently overlooked, though in some circumstances very important topic - concurrency handling in context of protection of the so-called *Domain Invariants*.

In other words, the question is the following: how to ensure that even in a multi-threaded environment we are able to always guarantee the immediate consistency for our business rules?

It's best to describe this problem with an example...

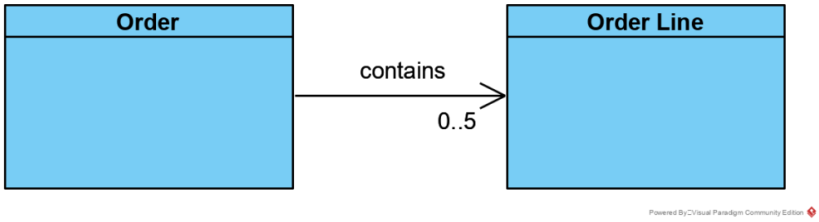
Problem

Let's assume that our domain has the concept of **Order** and **Order Line**. Domain expert said that:

One Order can have a maximum of 5 Order Lines.

In addition, he said that this rule must be met at all times (without exception). We can model it in the following

way:



Conceptual Model

The Aggregate

One of the main goals of our system is to enforce business rules. One way to do this is to use the Domain-Driven Design tactical building block - an *Aggregate*. The Aggregate is a concept created to enforce business rules (invariants). Its implementation may vary depending on the paradigm we use, but In object-oriented programming, it is an object-oriented graph as Martin Fowler describes it:

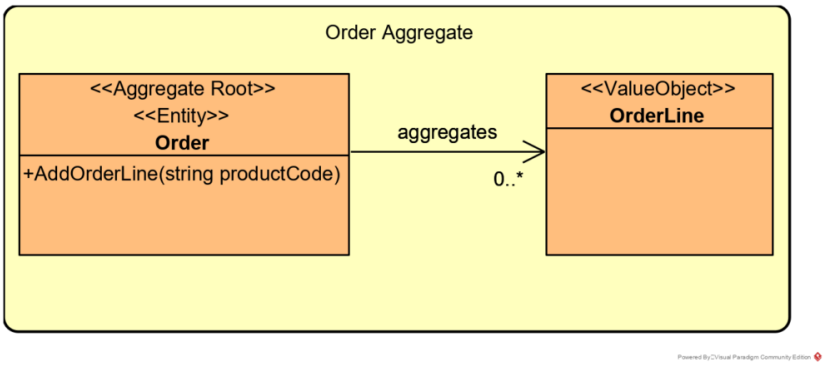
> A DDD aggregate is a cluster of domain objects that can be treated as a single unit.

And Eric Evans in DDD reference describes:

> Use the same aggregate boundaries to govern transactions and distribution. Within an aggregate boundary, apply consistency rules synchronously

Back to the example. How can we ensure that our Order will never exceed 5 Order Lines? We must have an object that will take care of it. To do this, it must have information about the current number of Order Lines. So it must have a state and based on this state its responsibility is to decide whether invariant is broken or not.

In this case, the Order seems to be the perfect object for this. It will become root of our Aggregate, which will have Order Lines. It will be his responsibility to add the order line and check that the invariant has not been broken.



Order Aggregate

Let's see how the implementation of such a construct may look like:

```
// Order Entity
public class Order : AggregateRootBase
{
    public Guid Id { get; private set; }

    private List<OrderLine> _orderLines;

    private DateTime? _modifyDate;

    private Order()
    {
```

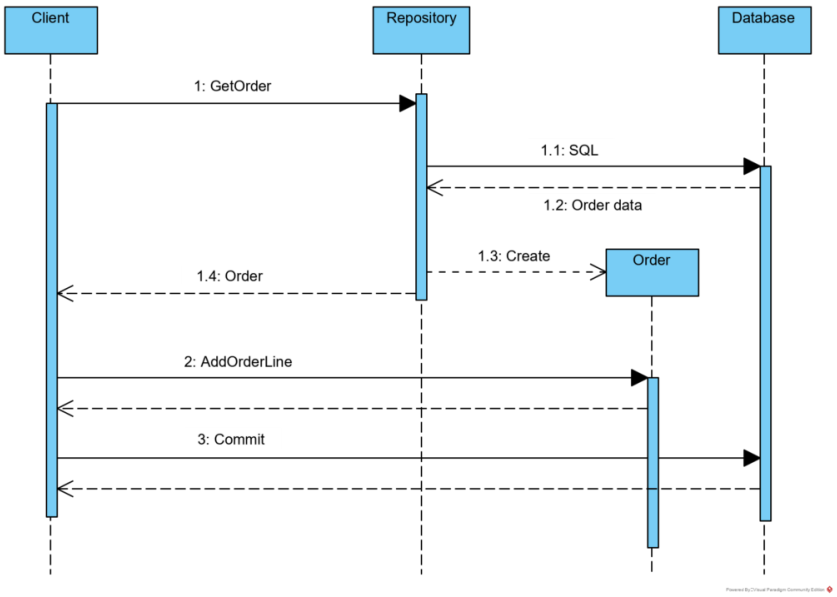
```
            _orderLines = new List<OrderLine>();
        }

        public void AddOrderLine(string productCode)
        {
            if (_orderLines.Count >= 5)
            {
                throw new Exception("Order cannot have
more than 5 order lines.");
            }

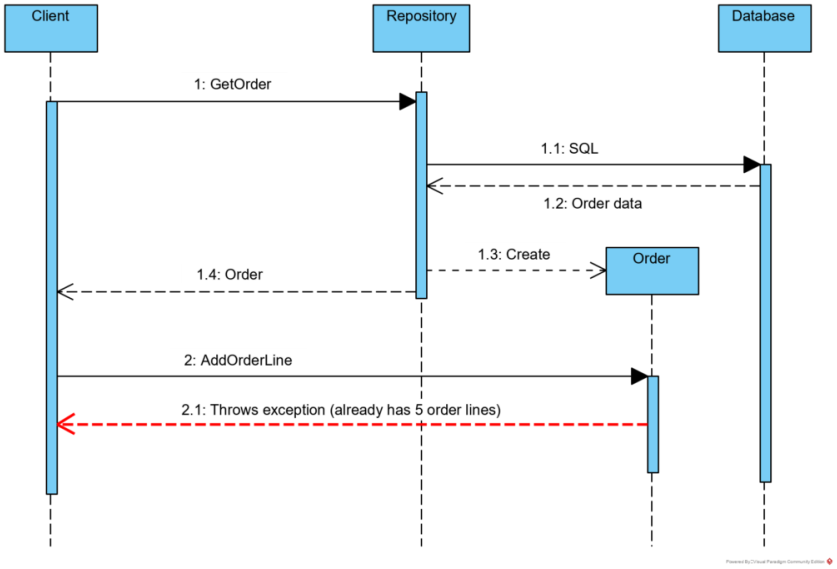

_orderLines.Add(OrderLine.CreateNew(productCode));
            _modifyDate = DateTime.Now;

            AddDomainEvent(new
OrderLineAddedDomainEvent(this.Id));
        }
}
```

Everything's fine right now, but this is only a static view of our model. Let's see what the typical system flow is when invariant is not broken and when is broken:



Process of adding Order Line - success



Process of adding Order Line - rule broken

Simplified implementation bellow:

```
// Add Order Line
[ApiController]
[Route("[controller]")]
public class OrdersController : ControllerBase
{
    private readonly OrdersContext _ordersContext;

    public OrdersController(OrdersContext
ordersContext)
    {
        _ordersContext = ordersContext;
    }

    [HttpPost]
    public async Task<IActionResult>
AddOrderLine(AddOrderLineRequest request)
    {
        var orderId = Guid.Parse("33d4201c-4a8e-
40a2-ae1d-50bc64097085");

        var order = await
_ordersContext.Orders.FindAsync(orderId);
```

```
            Thread.Sleep(3000);

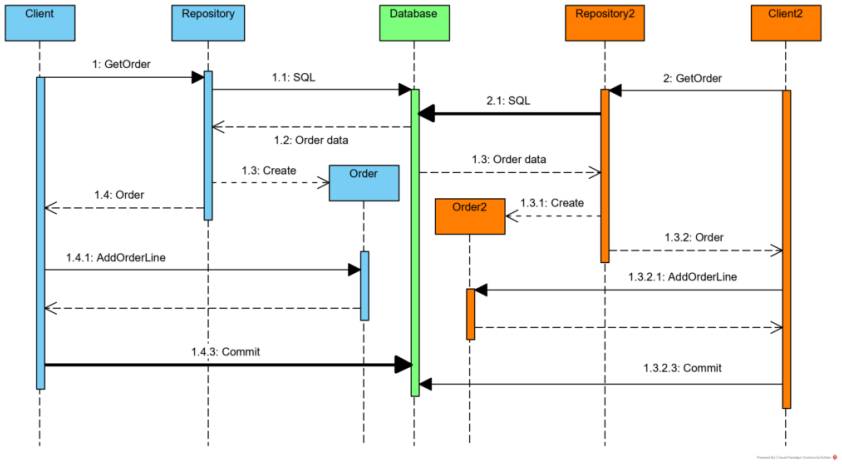
            order.AddOrderLine(request.ProductCode);

            await _ordersContext.SaveChangesAsync();

            return Ok();
        }
    }
```

Concurrency problem

Everything works nice and clean if we operate in a not heavy loaded environment. However, let's see what can happen when **2 threads almost at the same time** want to perform our operation:



Process of adding Order Line - business rule broken

As you can see in the diagram above, **2 threads load exactly the same aggregate at the same time**. Let's assume that the Order has **4** Order Lines. The aggregate with 4 Order Lines will be loaded in both the first and second threads. The exception will not be thrown, because of 4 < 5. Finally, depending on how we persist the aggregate, the following scenarios are possible:

a) If we have a relational database and a separate table for Order Lines then **2** Order Lines will be added giving a total of **6** Order Lines - the business rule is broken. b) If we store aggregate in one atomic place (for example in document database as a JSON object), the second thread will override the first operation and we (and the User) won't even know about this.

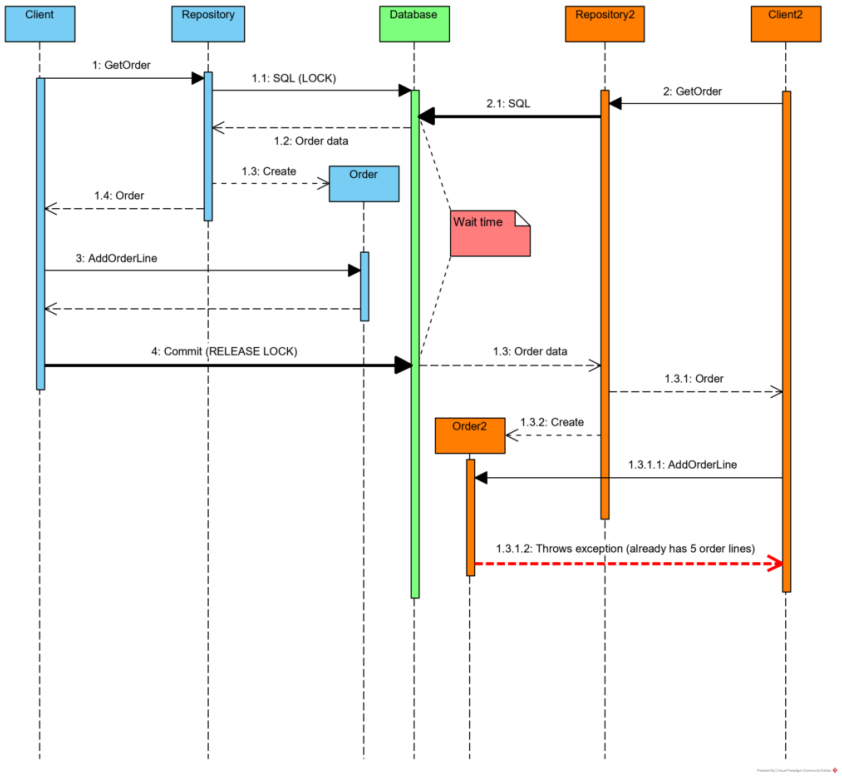
The reason for this behavior is that the second thread read data from the database (point 2.1) before the first one committed it (point 1.4.3).

Let's see how we can solve this problem.

Solution

Pessimistic concurrency

The first way to ensure that our business rule will not be broken is to use *Pessimistic concurrency*. In that approach, we allow **only one thread to process a given Aggregate**. This leads to the fact that the processing thread must block the reading of other threads by creating a lock. Only when the lock is released, the next thread can get the object and process it.



Pessimistic concurrency

The main difference from the previous approach is that the second thread waits for the previous one to finish (time between points 2.1 and 1.3). This approach causes us to lose performance because we can process transactions only one after the other. Moreover, it can lead to deadlocks.

How can we implement this behavior using EntityFramework Core and SQL server?

Unfortunately, EF Core does not support *Pessimistic Concurrency*. However, we can do it easily by ourselves using raw SQL and the query hint mechanism of the SQL Server engine.

Firsty, the database transaction must be set. Then the lock must be set. This can be done in two ways - either read data with query hint (XLOCK, PAGELOCK) or by updating the record at the beginning:

```
// Pessimistic concurrency
public async Task<IActionResult>
AddOrderLine(AddOrderLineRequest request)
{
    var orderId = Guid.Parse("33d4201c-4a8e-40a2-
ae1d-50bc64097085");

    await using (var tran = await
_ordersContext.Database.BeginTransactionAsync())
    {
        await _ordersContext.Database
            .ExecuteSqlRawAsync($"UPDATE
orders.Orders WITH (XLOCK) SET Id = Id  WHERE Id =
'{orderId}'");

        var order = await
_ordersContext.Orders.FindAsync(orderId);

        Thread.Sleep(3000);

        order.AddOrderLine(request.ProductCode);

        await _ordersContext.SaveChangesAsync();
    }

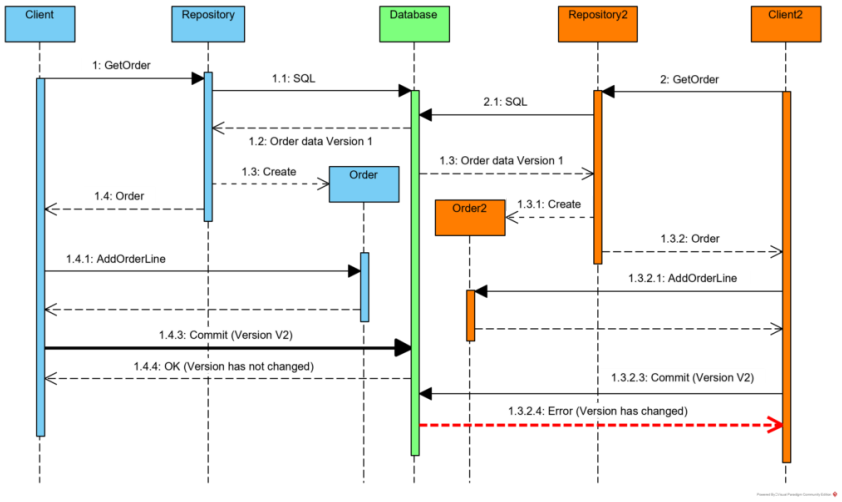
    return Ok();
}
```

In this way, the transaction on the first thread will receive a *Exclusive Lock* (on write) and until it releases it (through the transaction commit) no other thread will be able to read this record. Of course, assuming that our queries operate in at least read committed transaction isolation level to avoid the so-called dirty reads.

Optimistic concurrency

An alternative, and most often preferred solution is to use *Optimistic Concurrency*. In this case, the whole

process takes place without locking the data. Instead, the data in the database is versioned and during the update, it is checked - whether there was a change of version in the meantime.



Optimistic Concurrency

What does the implementation of this solution look like? Most ORMs support *Optimistic Concurrency* out of the box. It is enough to indicate which fields should be checked when writing to the database and it will be added to the `WHERE` statement. If it turned out that our statement has updated 0 records, it means that the version of the record has changed and we need to do a rollback. Though often the current fields are not used to check the version and special column called “Version” or “Timestamp” is added.

Back to our example, does adding a column with a version and incrementing it every time the Order entity is changed solve the problem? Well no. The aggregate must be treated as a whole, as a boundary of transaction and consistency.

Therefore, **the version incrementation must take place when we change anything in our aggregate**. If we have added the Order Line and we keep it in a separate table, ORM support for optimistic concurrency will not help us because it works on updates and here there are inserts and deletes left.

How can we know that the state of our *Aggregate* has changed? If you follow this blog well, you already know the answer - by Domain Events. If a Domain Event was thrown from the Aggregate, it means that the state has changed and we need to increase Aggregate version.

The implementation can look like this.

First, we add a `version` field to each Aggregate Root and the method to increment that version.

```
// AggregateRootBase with version
public class AggregateRootBase : Entity,
IAggregateRoot
{
    private int _versionId;

    public void IncreaseVersion()
    {
        _versionId++;
    }
}
```

Secondly, we add a mapping for the version attribute and indicate that it is a EF Concurrency Token:

```
// Order Entity Type Configuration Mapping
internal sealed class OrderEntityTypeConfiguration :
IEntityTypeConfiguration<Order>
{
```

```
    public void Configure(EntityTypeBuilder<Order>
builder)
    {
        builder.ToTable("Orders", "orders");

        builder.HasKey(b => b.Id);

builder.Property("_modifyDate").HasColumnName("Modif
yDate");

builder.Property("_versionId").HasColumnName("Versio
nId").IsConcurrencyToken();

        builder.OwnsMany<OrderLine>("_orderLines",
orderLine =>
        {

orderLine.WithOwner().HasForeignKey("OrderId");

            orderLine.ToTable("OrderLines",
"orders");

            orderLine.Property<Guid>
("Id").ValueGeneratedNever();
            orderLine.HasKey("Id");
            orderLine.Property<string>
("_productCode").HasColumnName("ProductCode");
        });
    }
}
```

The last thing to do is incrementing the version if any Domain Events have been published:

```
// Increase Order Aggregate version
[HttpPost]
public async Task<IActionResult>
AddOrderLine(AddOrderLineRequest request)
{
    var orderId = Guid.Parse("33d4201c-4a8e-40a2-
```

```
ae1d-50bc64097085");

    var order = await
_ordersContext.Orders.FindAsync(orderId);

    Thread.Sleep(3000);

    order.AddOrderLine(request.ProductCode);

    var domainEvents =
DomainEventsHelper.GetAllDomainEvents(order);

    if (domainEvents.Any())
    {
        order.IncreaseVersion();
    }

    await _ordersContext.SaveChangesAsync();

    return Ok();
}
```

With this setup, all updates will execute this statement:

```
info:
Microsoft.EntityFrameworkCore.Database.Command[20101
]
      Executed DbCommand (0ms) [Parameters=
[@p2='33d4201c-4a8e-40a2-ae1d-50bc64097085',
@p0='2020-05-14T21:02:41' (Nullable = true),
@p1='8', @p3='7'], CommandType='Text',
CommandTimeout='30']
      SET NOCOUNT ON;
      UPDATE [orders].[Orders] SET [VersionId] = @p1
      WHERE [Id] = @p2 AND [VersionId] = @p3;
      SELECT @@ROWCOUNT;
```

For our example, for second thread no record will be updated (`>@@ ROWOCOUNT = 0`), so EntityFramework will throw the following message:

Microsoft.EntityFrameworkCore.DbUpdateConcurren
Database operation expected to affect
1 row(s) but actually affected 0 row(s).
Data may have been modified or
deleted since entities were loaded. See
http://go.microsoft.com/fwlink/?
LinkId=527962 for information on
understanding and handling optimistic
concurrency exceptions.

and our Aggregate will be consistent - the 6th Order Line will not be added. The business rule is not broken, mission accomplished.

Summary

In summary, the most important issues here are:

- The Aggregate's main task is to protect invariants (business rules, the boundary of immediate consistency)
- In a multi-threaded environment, when multiple threads are running simultaneously on the same Aggregate, a business rule may be broken
- A way to solve concurrency conflicts is to use Pessimistic or Optimistic concurrency techniques
- *Pessimistic Concurrency* involves the use of a database transaction and a locking mechanism. In this way, requests are processed one after the other, so basically concurrency is lost and it can lead to deadlocks.

- *Optimistic Concurrency* technique is based on versioning database records and checking whether the previously loaded version has not been changed by another thread.
- Entity Framework Core supports *Optimistic Concurrency*. *Pessimistic Concurrency* is not supported
- The Aggregate must always be treated and versioned as a single unit
- Domain events are an indicator, that state was changed so Aggregate version should be changed as well

GitHub sample repository

Especially for the needs of this article, I created a repository that shows the implementation of 3 scenarios:

- without concurrency handling
- with Pessimistic Concurrency handling
- with Optimistic Concurrency handling

Link: https://github.com/kgrzybek/efcore-concurrency-handling.

COMMENTS

19 Comments

1 Login ▾

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

3 Share

Best Newest Oldest

F **Funky Monkey**

RELATED POSTS

SEE ALL BLOG POSTS>

Ignorance in mind using Entity Framework

READ MORE »

READ MORE »



© KAMIL GRZYBEK 2023

Software Engineering & Architecture

Privacy Policy